Text as Data

Class 2: Preprocessing and Feature Engineering

Petro Tolochko

CEU

Fall 2026

Last class

A document becomes a vector only after we choose its features.

Example: "Jobs, jobs, jobs!"

Before we count, we must decide:

- Is "Jobs" the same feature as "jobs"?
- Does the exclamation mark count?
- Do "job" and "jobs" belong together?

Today

- Choose preprocessing steps for a research question
- Explain what those choices preserve and discard
- Build word and phrase features
- Model word counts with a multinomial distribution
- Compare word use using odds and log odds
- Calculate and interpret TF-IDF weights
- Check whether a result changes under another reasonable pipeline

What is text preprocessing?

The steps we use to turn raw text into the units and features we will analyse.

Raw text
"Workers are demanding JOBS!"

Split into tokens
Workers | are | demanding | JOBS | !

Lowercase
workers | are | demanding | jobs | !

Optionally remove selected words and punctuation
workers | demanding | jobs

Each step changes what information remains available. Which steps we use depends on the research question.

Preprocessing is part of measurement

Question A: Which policy issues does a speech discuss?

Question B: How does the speaker express urgency?

Capital letters, repetition, and punctuation might matter differently for these questions.

The research question determines which distinctions we need.

A result can depend on preprocessing

Denny & Spirling (2018) examine preprocessing choices in unsupervised text analysis.

Preprocessing selects the features from which we infer patterns.

Every choice determines which distinctions the model can use:

- Removing a feature discards a distinction
- Combining forms treats them as equivalent
- Adding phrases makes some distinctions observable

Denny & Spirling (2018), *Political Analysis*, 26(2):168–189.

Seven choices, 128 representations

The paper varies seven on/off decisions:

1. Remove punctuation
2. Remove numbers
3. Lowercase
4. Stem
5. Remove stopwords
6. Add bigrams and trigrams
7. Remove terms in fewer than 1% of documents

$$2^7 = 128 \text{ preprocessing combinations}$$

Denny & Spirling (2018), *Political Analysis*.

What counts as a better representation?

Supervised analysis
We can compare predictions with held-out labels.

Unsupervised analysis
We must interpret the structure discovered from the chosen features.

Success at prediction does not guarantee useful discovery.

Denny & Spirling (2018), *Political Analysis*.

The same manifestos, different rankings

In the paper's Wordfish example:

UK party manifestos compared	8
Preprocessing combinations	128
Distinct estimated orderings	12

Preprocessing changed the estimated relative positions of the same documents.

Denny & Spirling (2018), *Political Analysis*.

A preprocessing workflow

1. Keep the raw text and document identifiers
2. Decide how to split the text into tokens
3. Decide which distinctions to combine or remove
4. Construct features: words, phrases, or other units
5. Count, filter, and weight the features
6. Inspect the result and compare alternatives

The order and the choices depend on the task.

Tokenization

Text: "We can't cut 2,000 public-sector jobs!"

Possible units include:

Decision	Possible tokens
Contraction	`can't` or `ca + n't`
Number	`2,000` or `2 + 000`
Hyphen	`public-sector` or `public + sector`

Token boundaries affect which expressions we can count.

Lowercasing

Before	After
`Jobs, jobs, JOBS`	`jobs`
`US, us`	`us`
`May, may`	`may`

Lowercasing combines variants, but can also merge different meanings.

Case can distinguish names, acronyms, and emphasis.

Punctuation and numbers

Text	Potential information
"No jobs!"	Emphasis
"A 5% wage increase"	Magnitude
"Article 5"	Legal reference
"public-sector jobs"	Compound expression

Remove, retain, or replace with a feature such as <NUMBER>.

A number placeholder retains presence but discards magnitude.

Stemming and lemmatization

Stemming: use rules to shorten word forms to a common stem.

`waiting, waits, waited` $\rightarrow$ `wait`

Lemmatization: identify a dictionary form using linguistic information.

`went, gone` $\rightarrow$ go

A stem need not be a dictionary word. A lemma can depend on context.

What does normalization remove?

`worker, workers` $\rightarrow$ `worker`

`worked, working` $\rightarrow$ `work`

Combining forms may help count a broad concept.

It can also remove distinctions in tense or number.

Tense and number may be evidence when studying style.

Stopwords

A stopword list specifies tokens to remove.

Lists often contain frequent function words:

$$\{\text{the}, \text{and}, \text{of}, \text{is}, \ldots\}$$

A task-specific list might also remove boilerplate.

A frequent word is not automatically irrelevant.

Pronouns can be evidence about group identity or blame.

Removing negation changes the evidence

A: "We are cutting jobs."

B: "We are not cutting jobs."

Remove `we`, `are`, and `not`:

$$\text{A} \rightarrow \texttt{cutting jobs}$$

$$\text{B} \rightarrow \texttt{cutting jobs}$$

Attention to jobs survives; the difference between a claim and its denial does not.

Word n-grams

An n-gram is a sequence of n consecutive tokens.

Text: `we need more jobs`

Features	Sequences
Unigrams	we; need; more; jobs
Bigrams	we need; need more; more jobs
Trigrams	we need more; need more jobs

A sequence of L tokens has $L - n + 1$ n-gram positions when $L \geq n$.

Phrases add information and dimensions

```
minimum wage    public sector    not hiring
```

- Phrases preserve some local word order
- Unigrams and bigrams can be used together
- Adding phrases usually enlarges the vocabulary
- Longer phrases occur less often and are harder to estimate

Local word order is retained; the full sentence structure is not.

Order matters: stopwords and bigrams

Original tokens: `not in work`

Bigrams first:
`not in`, `in work`

Remove "in" first, then form bigrams:
`not work`

The second pipeline creates a pair that was not adjacent in the original text.

Specify whether phrases refer to original or retained-token adjacency.

Which difference do we want to measure?

1. `we support jobs`
2. `they support jobs`
3. `we oppose jobs`

Count cosine	Keep pronouns	Remove we/they
Documents 1 and 2	0.667	1.000
Documents 1 and 3	0.667	0.500

Would you remove pronouns to study stance? To study group identity?

Invented classroom example; lowercase unigram counts.

Collection frequency and document frequency

Three toy documents:

1. `policy jobs jobs`
2. `policy wages`
3. `policy wages prices`

Term	Total count	Documents containing it
policy	3	3
jobs	2	1
wages	2	2
prices	1	1

Document frequency counts documents, not repetitions.

Filtering the vocabulary

Using document frequency, we could:

- Drop terms occurring in fewer than two documents
- Drop terms occurring in every document

For our toy corpus, these rules keep only `wages`.

Document 1 then has no retained terms.

Filtering can remove the very feature the research question concerns.

Counts, rates, and weights

Representation	Question it can help answer
Count	How often is a term used?
Binary presence	Does the document contain it?
Count per token	How large a share is it?
TF-IDF	Which terms distinguish this document?

Weighting changes the interpretation of a matrix cell.

A probability model for word counts

Counts describe the words we observed.

A language model asks:

How probable is a text under a particular model of word use?

We start with a simple model: each word has a fixed probability, and tokens are drawn independently.

Word probabilities

Fix a vocabulary with K terms:

$$\{\text{banana}, \text{chocolate}, \text{fudge}, \text{ice-cream}\}.$$

A model assigns a probability to every term:

$$p = (p_1, \ldots, p_K), \qquad p_v \geq 0, \qquad \sum_{v=1}^{K} p_v = 1.$$

For a text of length n:

1. Draw a token using these probabilities
2. Repeat n times, with the same probabilities
3. Count how often each term occurs

One word: the binomial model

For each token, ask: **banana or something else?**

If p is the probability of banana and there are n tokens:

$$X \sim \text{Binomial}(n, p)$$

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

For $n = 3$, $p = 0.5$, exactly two bananas have probability:

$$\binom{3}{2} (0.5)^2 (0.5) = 0.375.$$

Keeping the other words separate

Binomial: two bananas and one "other" token.

Multinomial: which other word did we draw?

For (banana, chocolate, fudge, ice-cream), suppose

$$p = (0.5,\ 0.2,\ 0.2,\ 0.1).$$

Other word	Count vector	Probability
chocolate	$(2, 1, 0, 0)$	$3(0.5)^2(0.2) = 0.150$
fudge	$(2, 0, 1, 0)$	$3(0.5)^2(0.2) = 0.150$
ice-cream	$(2, 0, 0, 1)$	$3(0.5)^2(0.1) = 0.075$

Together: $0.150 + 0.150 + 0.075 = 0.375$, as before.

Same draws; we now keep every word category separate.

All words: the multinomial model

The count vector has one entry for each vocabulary term:

$$x = (x_1, \ldots, x_K), \qquad \sum_{v=1}^{K} x_v = n.$$

$$P(X = x \mid n, p) = \frac{n!}{\prod_{v=1}^{K} x_v!} \prod_{v=1}^{K} p_v^{x_v}$$

- $p_v^{x_v}$: probability contribution of term v
- $n! / \prod_v x_v!$: number of sequences with these counts
- With two categories, this reduces to the binomial

Estimating two word distributions

Pool the observed tokens for each speaker.

	banana	chocolate	fudge	ice-cream	Total
John	7	4	1	0	12
Mary	1	3	7	5	16

Estimate each probability by its token share:

$$\widehat{p}_{gv} = \frac{c_{gv}}{\sum_u c_{gu}}.$$

$$\widehat{p}_{\text{John}} = (7/12, 4/12, 1/12, 0),$$

$$\widehat{p}_{\text{Mary}} = (1/16, 3/16, 7/16, 5/16).$$

A new text under John's model

New text: chocolate chocolate banana banana

In our vocabulary order, $x = (2, 2, 0, 0)$ and $n = 4$.

$$P(X = x \mid \widehat{p}_{\text{John}}, n = 4) = \frac{4!}{2!\,2!\,0!\,0!} \left(\frac{7}{12}\right)^2 \left(\frac{4}{12}\right)^2$$

$$\approx 6 \times 0.03781 \approx 0.2269.$$

Six token orders share these counts; the coefficient adds their probabilities.

Comparing the two models

For the same new count vector $x = (2, 2, 0, 0)$:

Model	Probability of these counts
John	$6(7/12)^2(4/12)^2 \approx 0.2269$
Mary	$6(1/16)^2(3/16)^2 \approx 0.000824$

The observed counts are more likely under John's fitted model.

This is $P(\text{text} \mid \text{speaker model})$, not yet $P(\text{speaker} \mid \text{text})$.

Unseen words and smoothing

John's sample contains no ice-cream, so $\widehat{p}_{\text{ice-cream}} = 0$.

Any new text containing it gets probability zero under that estimate.

One simple alternative adds $\alpha > 0$ to each vocabulary count:

$$\widetilde{p}_v = \frac{c_v + \alpha}{m + \alpha K}, \qquad m = \sum_{v=1}^{K} c_v.$$

Here m is the training-token total; K is the vocabulary size.

With $\alpha = 1$, John's ice-cream probability becomes

$$\frac{0+1}{12+4} = \frac{1}{16}.$$

An unobserved word is not necessarily an impossible word.

What this language model assumes

- Vocabulary and tokenization are fixed
- Token probabilities stay constant within each model
- Each token draw is independent given those probabilities
- Word order does not affect the count-vector probability

It describes word composition, but not grammar or context.

Reordering the same tokens leaves the probability unchanged.

Distinctive words across groups

Which words are more characteristic of one group of texts?

Examples: outlets, speakers, or time periods.

Compare the share of tokens belonging to a term, not just its raw count.

Token shares account for differences in corpus size.

Probability and odds

Estimate a word's probability from its share of **all tokens**.

$$p = \frac{\text{word count}}{\text{total tokens}}$$

Odds compare its count with **all other tokens**.

$$\text{odds} = \frac{p}{1-p}$$

For 8 occurrences in 10 tokens: $p = 0.8$, odds $= 8/2 = 4$.

At $p = 0.5$, the odds are one-to-one.

Log odds

Take the natural logarithm of the odds:

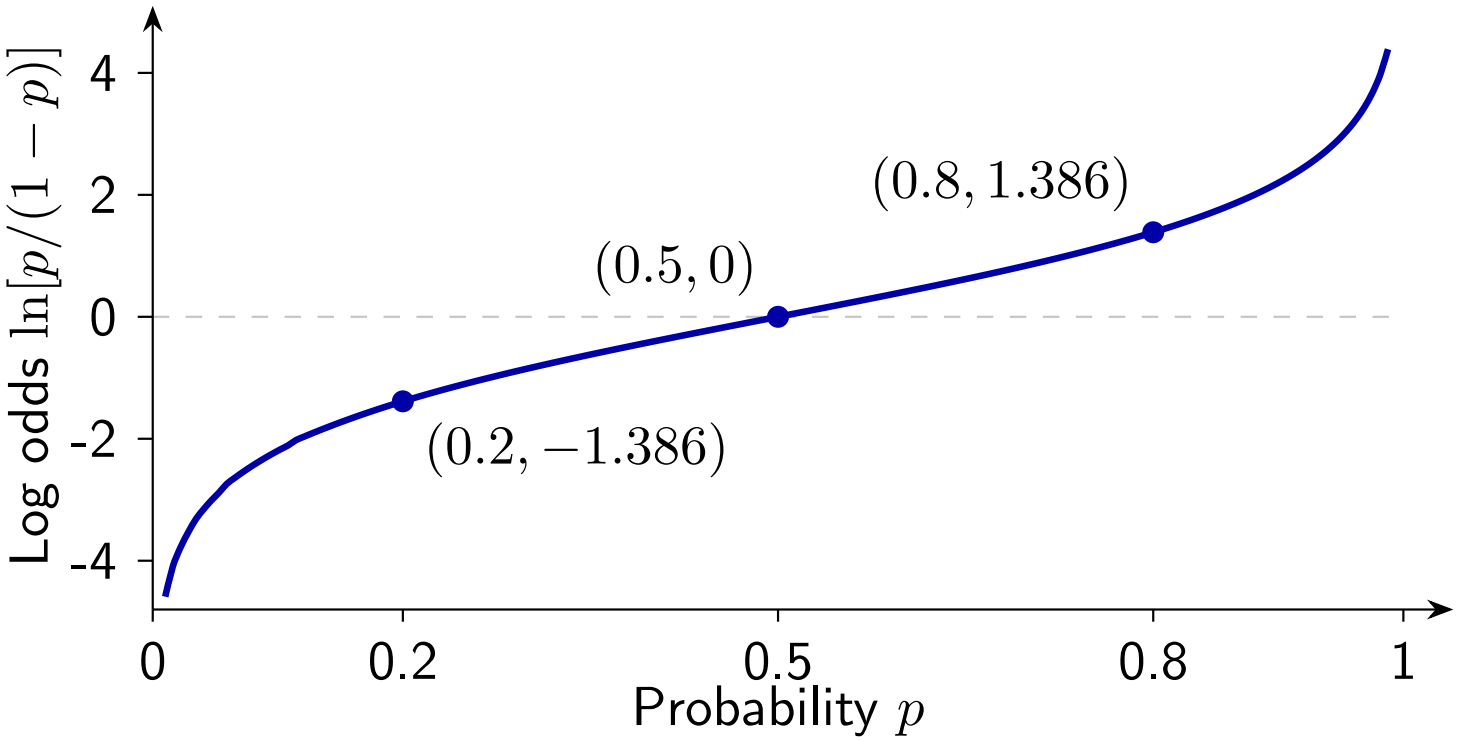
$$\text{logit}(p) = \ln\left(\frac{p}{1-p}\right)$$

Probability	Odds	Log odds
0.2	0.25	-1.386
0.5	1.00	0.000
0.8	4.00	1.386

Complementary probabilities have opposite log odds.

For $0 < p < 1$, log odds range over the whole real line.

Probability and log odds



Can a word have negative log odds in both groups and still be more characteristic of one?

Comparing log odds across groups

For the same word, estimate token probabilities $0 < p_A, p_B < 1$.

$$\Delta = \ln\left(\frac{p_A}{1-p_A}\right) - \ln\left(\frac{p_B}{1-p_B}\right)$$

$$\Delta = \ln\left(\frac{\text{odds}_A}{\text{odds}_B}\right)$$

- Positive: a larger token share in group A
- Negative: a larger token share in group B
- Zero: equal token shares

Comparing John and Mary

John uses "banana" 7 times in 12 tokens.

Mary uses "banana" 1 time in 16 tokens.

Odds compare banana tokens with **other tokens**:

$$\text{odds}_{\text{John}} = \frac{7/12}{5/12} = \frac{7}{5}, \qquad 12 - 7 = 5 \text{ others},$$

$$\text{odds}_{\text{Mary}} = \frac{1/16}{15/16} = \frac{1}{15}, \quad 16 - 1 = 15 \text{ others}.$$

$$\Delta = \ln\left(\frac{7}{5}\right) - \ln\left(\frac{1}{15}\right) = \ln(21) \approx 3.045$$

Banana has 21 times the odds under John's model.

Rare words and uncertainty

A word seen once in one group and never in another creates a problem: the raw log odds are not finite.

- Smoothing avoids infinite estimates
- Small counts can make estimates unstable
- Effect size and uncertainty are different things
- Inspect the documents behind distinctive words

A ranked word list is a starting point for interpretation.

TF-IDF: the idea

Term frequency (TF)
How often a term occurs within this document.

Inverse document frequency (IDF)
How uncommon the term is across documents in this corpus.

TF-IDF combines the two.

Return to our three-document corpus:

1. `policy jobs jobs`
2. `policy wages`
3. `policy wages prices`

Here, jobs distinguishes document 1; policy occurs everywhere.

TF-IDF: one explicit definition

For today's hand calculation, use raw counts as TF:

$$\mathrm{tf}_{dt} = \mathrm{count}(t, d)$$

$$\mathrm{idf}_t = \ln\left(\frac{N}{\mathrm{df}_t}\right) \qquad w_{dt} = \mathrm{tf}_{dt}\,\mathrm{idf}_t$$

- N: number of documents
- df_t: number of documents containing term t
- Natural logarithms; no smoothing or row normalization here

TF-IDF has several conventions. Always state which one you use.

Calculating inverse document frequency

Our corpus has $N = 3$ documents.

Term	df	$\ln(3/\text{df})$
policy	3	0.000
jobs	1	1.099
wages	2	0.405
prices	1	1.099

A term found in every document receives IDF $= 0$ under this definition.

Equal document frequencies produce equal IDF weights.

Weighting document 1

Document 1: `policy jobs jobs`

Term	TF	IDF	TF-IDF
policy	1	$\ln(3/3)$	0.000
jobs	2	$\ln(3)$	2.197
wages	0	$\ln(3/2)$	0.000
prices	0	$\ln(3)$	0.000

Multiply before rounding: $2\ln(3) = 2.197224\ldots \approx 2.197$.

A present term can receive zero weight when its IDF is zero.

The TF-IDF matrix

	policy	jobs	wages	prices
D_1	0	2.197	0	0
D_2	0	0	0.405	0
D_3	0	0	0.405	1.099

`jobs` distinguishes document 1; `prices` distinguishes document 3.

The weights depend on the other documents in the corpus.

The same text can receive different weights in a different corpus.

TF-IDF and document similarity

Using the same vocabulary:

Pair	Count cosine	TF-IDF cosine
D_1, D_2	0.316	0.000
D_1, D_3	0.258	0.000
D_2, D_3	0.816	0.346

With counts, every pair shares `policy`.
With these TF-IDF weights, that shared feature contributes zero.

Which similarity is more useful for your question?

Software conventions matter

The default scikit-learn convention uses:

$$\mathrm{idf}_t = \ln\left(\frac{1+N}{1+\mathrm{df}_t}\right) + 1$$

It multiplies by term counts, then normalizes each nonzero row to Euclidean length 1.

A term in every document has IDF $= 1$, not 0.

Different output from our hand calculation need not be a bug.

scikit-learn: TfidfTransformer documentation.

What TF-IDF cannot tell us

- A high weight does not establish that a concept is important
- A rare typo can receive a high weight
- A common but meaningful term can be downweighted
- A synonym with no shared features can still look unrelated

For employment attention, a validated count or rate may be easier to interpret than TF-IDF.

Choosing and documenting a pipeline

- State the research question and unit of analysis
- Record tokenization, normalization, and exact removal rules
- Report vocabulary size and empty documents
- Specify weighting and its reference corpus
- Read examples and compare plausible alternatives

For prediction, learn vocabulary and IDF on training data only.

Exercise: choose a representation

Text: "We are NOT cutting 200 public-sector jobs!"

In pairs, choose preprocessing for one question:

1. Does the text discuss employment?
2. Does the speaker claim that jobs are being cut?
3. How does the speaker express emphasis?

Specify what you keep, remove, or combine. Explain one risk.

Three minutes. Invented sentence.

Practical: two preprocessing choices

Use these invented statements:

1. We are creating jobs.
2. We are not creating jobs.
3. They are creating jobs!

Compare two stopword lists:

- A: `we, they, are`
- B: `we, they, are, not`

Practical: tokenize and count

```
import re
from collections import Counter

texts = ["We are creating jobs.",
         "We are not creating jobs.",
         "They are creating jobs!"]
stop = {"we", "they", "are"}

def tokens(text):
    return [w for w in re.findall(r"[a-z]+", text.lower())
            if w not in stop]

for text in texts:
    print(tokens(text), Counter(tokens(text)))
```

Practical: inspect the lost information

1. Run the code with stopword list A, then list B
2. Identify which documents become identical
3. Add bigrams of the retained tokens
4. Explain what those bigrams still fail to preserve

Write a short answer: which pipeline would you use for claims about job creation, and why?

Checking sensitivity on a real corpus

Return to the four State of the Union speeches from class 1.

Hold the documents fixed. Change one representation choice:

- Keep versus remove stopwords
- Unigrams versus unigrams plus bigrams
- Counts versus TF-IDF

Compare vocabulary size, top features, and similar documents. Read passages behind any change.

Class repository: `data/sotu_sample.csv`.

Sensitivity is a measurement diagnostic

Denny and Spirling's `preText` compares document-distance rankings across preprocessing choices.

Unusual rankings flag sensitivity; they do not identify a uniquely correct representation.

Denny & Spirling (2018), *Political Analysis*.

In our exercise:

1. Choose alternatives that make sense for the question
2. Inspect which document relationships change
3. Read those documents and explain the difference

Key takeaways

- Preprocessing determines which distinctions remain observable
- Multinomial probabilities describe counts under explicit assumptions
- Log-odds differences compare relative word use across groups
- TF-IDF weights depend on the reference corpus
- Sensitivity checks complement substantive validation

Next class: dictionaries, annotation, and turning a concept into labels.